

Name: Prudhvi Varma

CSE3068: Sequential and Spatial Data Mining

Slot: D1 + TD1 + TDD1

DA2

ST-SeqMiner: Implementation Architecture and Engineering of a Spatio-Temporal Trajectory and Sequential Movement Pattern Mining Engine

Abstract—This document details the engineering architecture, mathematical algorithms, and production implementation of ST-SeqMiner, a high-performance spatial and sequential telemetry data mining engine. Designed to process continuous, high-volume mobile check-in streams from Location-Based Social Networks (LBSNs), the engine integrates five foundational computational layers: continuous spatial discretization via uniform geographic tessellation, temporal trajectory sessionization with static stay-point deduplication, prefix-projected sequential pattern growth using PrefixSpan, density-based spatial hotspot clustering parameterized with Great-Circle Haversine metrics, and next-location movement prediction through 1st-Order and 2nd-Order Markov Transition Chains. The system is developed in modular Python 3, adhering strictly to asymptotic efficiency constraints and publication-grade software engineering standards. Complete architectural specifications, formal algorithmic formulations, data flow diagrams (DFD), and interface definitions are presented.

Keywords—*Spatio-Temporal Data Mining, PrefixSpan, Spatial Tessellation, Haversine DBSCAN, Markov Transition Modeling, Trajectory Sessionization.*

1. Introduction & Architectural Overview

Modern urban computing systems rely on spatio-temporal telemetry generated by mobile devices, GPS receivers, and location-aware social platforms. A raw trajectory trace is defined as a continuous sequence of time-stamped geographic coordinates:

$$T_u = \langle (\phi_1, \lambda_1, t_1), (\phi_2, \lambda_2, t_2), \dots, (\phi_m, \lambda_m, t_m) \rangle \quad (1)$$

where $\phi_i \in [-90.0^\circ, 90.0^\circ]$ denotes latitude, $\lambda_i \in [-180.0^\circ, 180.0^\circ]$ denotes longitude, and $t_i \in \mathbb{R}^+$ represents the UTC epoch timestamp. Extracting recurring movement semantics from raw telemetry introduces three fundamental computational complexities: continuous state spaces in $\mathbb{R}^2$, combinatorial path explosion scaling as $\mathcal{O}(|\Sigma|^L)$, and stochastic transition dynamics. ST-SeqMiner resolves these challenges through an end-to-end data processing pipeline that systematically transforms noisy raw telemetry into discrete sequential association rules, spatial density hotspots, and transition probability matrices.

2. Mathematical Formulations & Algorithmic Specifications

2.1 Spatial Grid Tessellation (Discretization)

Continuous coordinates $p_i = (\phi_i, \lambda_i)$ are mapped into discrete spatial state identifiers $S(i_{\text{lat}}, i_{\text{lon}})$ using uniform geographic grid binning:

$$i_{\text{lat}} = \lfloor \phi_i / \Delta \rfloor, i_{\text{lon}} = \lfloor \lambda_i / \Delta \rfloor \Rightarrow S_i = S_{\lfloor \cdot \rfloor} + i_{\text{lat}} + \lfloor \cdot \rfloor + i_{\text{lon}} \quad (2)$$

where $\Delta = 0.02^\circ$ represents the spatial resolution grid step (~ 2.22 km at the equator). Each discrete state S_i retains a deterministic spatial centroid $(\bar{\phi}, \bar{\lambda})$:

$$\bar{\phi} = (i_{\text{lat}} + 0.5) \cdot \Delta, \bar{\lambda} = (i_{\text{lon}} + 0.5) \cdot \Delta \quad (3)$$

Algorithm 1: Spatial Grid Discretization

Input : Coordinate tuple (phi, lambda), resolution parameter Delta

Output: State identifier S_cell, Centroid tuple (phi_c, lambda_c)

```

1: i_lat <- floor(phi / Delta)
2: i_lon <- floor(lambda / Delta)
3: S_cell <- 'S_' + str(i_lat) + '_' + str(i_lon)
4: phi_c <- (i_lat + 0.5) * Delta
5: lambda_c <- (i_lon + 0.5) * Delta
6: return S_cell, (phi_c, lambda_c)

```

2.2 Trajectory Sessionization & Delta-t Filtering

Telemetry streams often feature prolonged static dwelling where a client logs repeated check-ins from a stationary location. To isolate genuine displacement, consecutive identical states ($S_t = S_{t+1}$) are collapsed into a single visit. Furthermore, when elapsed idle time exceeds threshold $\Delta t_{\text{max}} = 12.0$ hours, the stream is partitioned into independent trip sessions:

$$\Delta t = t_{k+1} - t_k > \Delta t_{\text{max}} \Rightarrow \text{New Trajectory Session} \quad (4)$$

Trajectories satisfying $2 \leq |S| \leq 25$ are appended to the sequential transaction database $\mathcal{D}$.

Algorithm 2: Trajectory Extraction and Sessionization

Input : Sorted user records $[(S_1, t_1), \dots, (S_m, t_m)]$, Delta_t_max, [L_min, L_max]

Output: Set of trip trajectory sessions Traj(u)

```

1: Traj(u) <- empty list; current_session <- empty list
2: last_time <- null; last_cell <- null
3: for each (S_k, t_k) in records do
4:   if last_time != null and (t_k - last_time) > Delta_t_max then
5:     if length(current_session) >= L_min then
6:       append current_session[0 : L_max] to Traj(u)
7:       current_session <- empty list; last_cell <- null
8:   if S_k != last_cell then
9:     append S_k to current_session; last_cell <- S_k
10:  last_time <- t_k
11: if length(current_session) >= L_min then append current_session[0 : L_max] to
    Traj(u)
12: return Traj(u)

```

2.3 PrefixSpan Pattern Growth Algorithm

Given transaction database $\mathcal{D}$ and candidate sequence α , the support of α is defined as:

$$\text{Support}(\alpha) = \frac{|\{S \in \mathcal{D} | \alpha \subseteq S\}|}{|\mathcal{D}|} \quad (5)$$

PrefixSpan operates via recursive prefix-projected database growth, completely avoiding the candidate generation overhead of Apriori-like methods. Sequential association rules $\alpha \rightarrow \beta$ are derived with Support, Confidence, and Lift:

$$\text{Confidence}(\alpha \rightarrow \beta) = \frac{\text{Support}(\alpha \circ \beta)}{\text{Support}(\alpha)} \quad (6)$$

$$\text{Lift}(\alpha \rightarrow \beta) = \frac{\text{Confidence}(\alpha \rightarrow \beta)}{\text{Support}(\beta)} \quad (7)$$

2.4 Density-Based Spatial Clustering (Haversine DBSCAN)

To identify geographic mobility hubs independent of grid discretization, DBSCAN is executed over spherical coordinates using Great-Circle Haversine distance:

$$d_{\text{hav}}(p_1, p_2) = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1)\cos(\phi_2)\sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (8)$$

where $R = 6,371.0$ km, $\varepsilon = 2.0$ km, and $\text{MinPts} = 40$ points. Points lacking neighborhood density are labeled as spatial noise (-1) .

2.5 Markovian Trajectory Transition Modeling

Human trajectory transitions are modeled using 1st-Order and 2nd-Order Markov Transition Chains:

$$P(S_{t+1} = S_j \mid S_t = S_i) = \frac{C(S_i \rightarrow S_j)}{\sum_k C(S_i \rightarrow S_k)} \quad (9)$$

$$P(S_{t+1} = S_j \mid S_t = S_i, S_{t-1} = S_h) = \frac{C(S_h, S_i \rightarrow S_j)}{\sum_k C(S_h, S_i \rightarrow S_k)} \quad (10)$$

If context (S_{t-1}, S_t) is unobserved during testing, the model gracefully backs off to the 1st-order transition distribution. Model performance is evaluated on held-out transitions using Top- k Accuracy ($k \in \{1, 3, 5\}$) and Mean Reciprocal Rank (MRR):

$$\text{MRR} = \frac{1}{|T|} \sum_{i=1}^{|T|} \frac{1}{\text{Rank}_i} \quad (11)$$

3. System Architecture & Data Flow Design (DFD)

The complete data processing workflow is organized as a clean top-to-bottom pipeline connecting external entities, sequential transformation processes, and persistent repositories.



Figure 1: Level-0 & Level-1 Data Flow Diagram for ST-SeqMiner. Illustrates the clean sequential hierarchy connecting mobile telemetry input, ingestion, spatial discretization, parallel mining engines (P3.1–P3.3), persistent stores (D1–D5), and downstream analyst delivery.

4. Software Implementation & Modular Architecture

The production software is encapsulated in `DA2/spatial_sequential_miner.py`, structured into six modular classes:

- `SpatioTemporalDataLoader`: TSV ingestion, coordinate validation, and chronological sorting. Complexity: $\mathcal{O}(N \log N)$.
- `SpatialDiscretizer`: Discretization into grid cells and reverse centroid resolution. Complexity: $\mathcal{O}(N)$.

- TrajectoryExtractor: Static stay deduplication and trip sessionization. Complexity: $\mathcal{O}(N)$.
- PrefixSpanMiner: Pure-Python prefix-projected pattern growth mining. Complexity: $\mathcal{O}(|\mathcal{D}| \cdot L_{max} \cdot |P|)$.
- SpatialDensityClusterer: Spherical DBSCAN with BallTree Haversine metric. Complexity: $\mathcal{O}(M \log M)$.
- MarkovTrajectoryPredictor: 1st/2nd-order Markov transition matrices and evaluation. Complexity: $\mathcal{O}(|\mathcal{D}| \cdot \mu_L)$.

5. Execution Protocol & CLI Reference

The complete pipeline is executed from the terminal using the command-line interface:

```
python DA2/spatial_sequential_miner.py \
  --dataset "data/loc-brightkite_totalCheckins.txt" \
  --sample-size 150000 \
  --delta-deg 0.02 \
  --min-sup 0.008 \
  --eps-km 2.0 \
  --output-dir "DA2"
```

Table 1: ST-SeqMiner Command-Line Interface Parameters

Parameter	Type	Default	Description
--dataset	string	data/loc-brightkite_totalCheckins.txt	Path to raw Stanford SNAP Brightkite TSV file.
--sample-size	integer	150000	Telemetry record sampling bound.
--delta-deg	float	0.02	Spatial grid resolution in degrees (~ 2.22 km).
--min-sup	float	0.008	PrefixSpan minimum relative support threshold (0.8%).
--eps-km	float	2.0	DBSCAN Haversine neighborhood radius (kilometers).
--output-dir	string	DA2	Target directory for benchmark metrics and logs.

References

- [1] J. Han, J. Pei, B. Mortazavi-Asl, Q. Chen, U. Dayal, and M.-C. Hsu, 'PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth,' in Proc. 17th Int. Conf. on Data Engineering (ICDE), 2001, pp. 215–224.
- [2] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, 'A Density-Based Algorithm for Discovering Clusters in Large

Spatial Databases with Noise,' in Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD), 1996, pp. 226–231.

[3] E. Cho, S. A. Myers, and J. Leskovec, 'Friendship and Mobility: User Movement in Location-Based Social Networks,' in Proc. 17th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD), 2011, pp. 1082–1090.

[4] Y. Zheng, 'Trajectory Data Mining: An Overview,' ACM Transactions on Intelligent Systems and Technology (TIST), vol. 6, no. 3, pp. 29:1–29:41, 2015.

[5] C. Cheng, H. Yang, I. King, and M. R. Lyu, 'Fused Matrix Factorization with Geographical and Social Influence in Location-Based Social Networks,' in Proc. 26th AAAI Conf. on Artificial Intelligence (AAAI), 2012, pp. 17–23.